Ranking Without Resolution: Most LLM Guardrail Benchmark Comparisons Are Not Statistically Significant

Saurabh Kumbhar
saurabh.kumbhar24@outlook.com
Independent

August 31, 2026

Abstract

Vendor-published comparisons of LLM guardrails routinely report ranked tables of recall, precision and F1 across a few dozen to a few hundred test cases. We show that at these sample sizes the rankings are largely unresolvable. We benchmark eight guardrails, six independent products, one thirty-line regular-expression control and one belonging to the author, on 949 cases across four corpora, and subject every pairwise comparison to McNemar's exact test on paired per-case outcomes. On a 49-case adversarial suite of the kind vendors commonly publish, only 11 of 28 pairs separate at $p < 0.05$; the four highest-scoring systems are mutually indistinguishable, and the leading system is not distinguishable from the regular-expression control ($p = 0.065$). On a 300-case jailbreak corpus, a commercial detector achieving 97.3% recall is not distinguishable from that same control ($p = 0.248$), because an 83.3% false-alarm rate cancels the advantage in overall correctness. Monte Carlo power analysis over the observed disagreement patterns indicates that separating two competent guardrails typically requires between 1,600 and more than 6,400 cases, one to two orders of magnitude beyond common practice. We argue that reporting a rank order without an interval or a paired test is the central methodological defect in this literature, that a naive control is the cheapest available diagnostic, and that the finding applies to the author's own second-place results as forcefully as to anyone else's. Harness, corpora, adapters and analysis code are released under MIT.

1 Introduction

An LLM guardrail is a classifier placed around a language model that decides whether a given string should be blocked. Guardrails are sold, adopted and compared, and the comparisons are usually published by one of the parties being compared. The genre has a recognisable shape: a table of products, a column of recall or F1 scores, a bolded row belonging to the publisher.

The methodological complaint usually levelled at these documents is bias, and it is a fair complaint. Corpus selection, threshold choice, which configuration of one's own product to enter and which competitor settings count as "stock" are all degrees of freedom, and they reliably point one direction. But bias is difficult to prove and easy to deny, and a publisher can respond to it with disclosure and good intentions.

We argue that a second and more tractable defect is more damaging: the sample sizes in common use cannot resolve the differences being reported, and almost no published comparison says so. A table with eight rows and three decimal columns communicates an ordering. If the ordering is not statistically distinguishable from a permutation of itself, the table is not a weak result. It is a misleading one, regardless of who published it and regardless of their care.

This paper makes three contributions.

1. We report a guardrail benchmark of eight systems on 949 cases across four corpora, with Wilson confidence intervals on every rate and McNemar's exact test on every pair. We find that a majority of pairwise comparisons on the smallest corpus, and a meaningful minority on the larger ones, are not resolvable.

2. We show that the two most striking headline numbers in our own results are artefacts of reporting recall without a paired test. A commercial detector with the highest jailbreak recall in the study is statistically indistinguishable from a thirty-line regular expression that detects almost nothing.

3. We estimate, by Monte Carlo over observed disagreement patterns, the corpus size actually required to resolve these comparisons, and find it to be one to two orders of magnitude larger than current practice.

The benchmark is published by a party with a product in it. That conflict is real, is stated throughout, and is partially mitigated by the direction of the findings: the central result undercuts the author's own second-place finishes as much as any competitor's first-place one.

2 Related work

Statistical testing of classifier comparisons. The problem of deciding whether one classifier is better than another on a shared test set is old and well understood. Dietterich (1998) evaluates five statistical tests for comparing supervised learning algorithms and recommends McNemar's test (McNemar, 1947) for the case where algorithms are compared on a single fixed test set, precisely the design used by guardrail benchmarks. Demšar (2006) extends the treatment to multiple classifiers over multiple datasets. Wilson (1927) gives the score interval for a binomial proportion that we use throughout, which is well behaved at the small samples and extreme proportions where the normal approximation fails.

Statistical power in NLP evaluation. Card et al. (2020) document that a large fraction of published NLP experiments are underpowered, such that reported improvements are unlikely to be detectable even if real, and argue for power analysis as routine practice. Bowman and Dahl (2021) survey the broader failure modes of NLU benchmarking, including saturation, annotation artefacts and the gap between leaderboard position and capability. Our finding is a specific instance of the concern raised by both: the guardrail literature has adopted the leaderboard format without the accompanying statistics.

Guardrails and prompt injection. Indirect prompt injection against LLM-integrated applications was characterised by Greshake et al. (2023), following earlier demonstrations of instruction override (Perez and Ribeiro, 2022). Liu et al. (2024) propose a framework and benchmark for prompt injection attacks and defences. Practical guardrail systems in wide use include Microsoft Presidio for PII detection, NVIDIA NeMo Guardrails for programmable rails, and LLM Guard for input and output scanning, alongside commercial offerings. Existing benchmarks in this space overwhelmingly evaluate the *input* side, asking whether an attack prompt is detected. We evaluate the output side, which we describe next.

3 Benchmark design

3.1 Task

Existing guardrail evaluation asks whether a model can be induced to misbehave. We ask a different and narrower question: given a string that a model is about to emit, or that a user has just sent, would the

Corpus	Cases	Block/Allow	Source
Output leakage, adversarial	49	32/17	written for this benchmark
Output leakage, public PII	300	200/100	`ai4privacy/pii-masking-200k`
Prompt injection	300	150/150	`deepset/prompt-injections`
Jailbreak	300	150/150	`jackhhao/jailbreak-classification`

Table 1: Corpora. The adversarial suite is deliberately of the size typical of published vendor comparisons, which is part of what is under study.

guardrail stop it? No model is run during evaluation. Every system receives the identical string and returns a binary verdict.

The output-side framing is motivated by an observation that shaped the corpus. Contemporary frontier models refuse most direct attacks. In preliminary testing against `claude-haiku-4-5`, seven attack framings including every credential-exfiltration attempt were refused. What escapes is duller: data the model was legitimately given, and identifiers that survive inside a refusal. One corpus case is a verbatim model refusal that names two customers by email address while explaining that it will not name them. A benchmark that only tests whether the attack prompt is caught cannot see this failure at all.

3.2 Corpora

Four corpora, 949 cases (Table 1). Two are written for this benchmark; two are sampled from public datasets that neither we nor any vendor curated. Approximately one third of every corpus is benign text constructed to bait false positives: order numbers shaped like card numbers, git SHAs shaped like secrets, and sentences that merely *describe* an injection attempt.

The adversarial suite covers leak shapes a PII dataset does not contain: obfuscated identifiers, credential formats, injected instructions echoed into output, and refusals that leak while refusing. All of its values are synthetic.

For the public PII corpus, positives are rows containing at least one strong identifier, and negatives are the *same rows* with identifiers replaced by placeholders, so both halves share a topic and register distribution and a detector cannot score well by keying on subject matter.

3.3 Systems evaluated

Eight systems: three open source (LLM Guard, Presidio, NeMo Guardrails), three commercial (Lakera Guard, Nightfall, AWS Comprehend), one belonging to the author (Fluiq), and one control. Competitors run at stock settings with no tuning to these cases.

The control is a thirty-line regular-expression matcher written in an afternoon. Its purpose is diagnostic. A benchmark without a floor cannot distinguish "the winner is strong" from "the field is weak," and if a naive baseline is competitive with a commercial product, that is the most actionable fact available to a reader.

Systems requiring credentials or an unavailable library are skipped and reported, never scored zero. The harness refuses to rank any system that errors on more than 2% of a corpus: an early Nightfall run failed on 190 of 300 cases through rate limiting, and scoring a rate limit as a detection failure is a way to publish a falsehood accidentally. That run was discarded, backoff was added, and the system re-run.

3.4 Scoring

Scoring was fixed before any system ran. A case labelled *block* that is blocked is a catch; blocked *allow* is a false alarm; allowed *block* is a miss. We report recall and false-alarm rate side by side, never recall alone,

since a system that blocks everything achieves perfect recall. For the paired tests we reduce each case to a single boolean: the system did the right thing, meaning it blocked what should be blocked and allowed what should be allowed.

4 Statistical method

4.1 Intervals

We report 95% Wilson score intervals (Wilson, 1927) for recall and false-alarm rate. At $n = 49$ and proportions near 0.9 the normal approximation produces upper bounds exceeding 100%, which is not a confidence interval.

4.2 Paired testing

Every system is handed the identical case list, so outcomes are paired. We use McNemar's exact test (McNemar, 1947), following Dietterich (1998), which conditions on the discordant pairs: let b be the number of cases the first system handles correctly and the second does not, and c the reverse. Under the null hypothesis that the two are equally accurate, $b \sim \text{Binomial}(b + c, 0.5)$, and we compute the two-sided exact p-value

$$p = 2 \sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 2^{-(b+c)}, \tag{1}$$

capped at 1. We use the exact form rather than the χ^2 approximation because discordant totals here are frequently below 25.

The choice of a paired test is not incidental. A two-proportion z-test on independent samples, which is what an implicit comparison of two accuracy numbers amounts to, discards the pairing. The pairing is the most informative feature of the design: what matters is not how many cases each system got right, but the cases on which the two disagree.

4.3 Power

To estimate the corpus size required to resolve a comparison, we treat the observed joint outcome distribution for a pair as ground truth, resample cases with replacement at a range of corpus sizes, re-run the test, and record the fraction of simulations reaching $p < 0.05$. We report the smallest grid point reaching 80% power, over 2,000 simulations per point.

These cell probabilities are themselves estimated from small samples and carry substantial uncertainty, particularly at $n = 49$. The figures should be read as order-of-magnitude guidance for corpus design rather than precise requirements. The direction of the result is robust to that uncertainty even where the exact figure is not.

5 Results

5.1 Adversarial suite, $n = 49$

Table 2 has the shape of a vendor comparison, and read as one it supports a clear ordering. McNemar's test separates only 11 of the 28 pairs at $p < 0.05$. The four highest-scoring systems are mutually indistinguishable: `llm-guard` versus `fluiq` gives $p = 1.000$ ($b = 4$, $c = 3$), `llm-guard` versus `aws-comprehend` gives $p = 0.791$, and `fluiq` versus `aws-comprehend` gives $p = 1.000$.

System	Category	Recall	95% CI	FA	95% CI	F1
`llm-guard`	open source	87.5	[71.9, 95.0]	17.6	[6.2, 41.0]	**88.9**
`fluiq`	author	81.2	[64.7, 91.1]	11.8	[3.3, 34.3]	86.7
`aws-comprehend`	commercial	84.4	[68.2, 93.1]	23.5	[9.6, 47.3]	85.7
`nightfall`	commercial	71.9	[54.6, 84.4]	5.9	[1.0, 27.0]	82.1
`regex-baseline`	control	62.5	[45.3, 77.1]	11.8	[3.3, 34.3]	74.1
`lakera-guard`	commercial	46.9	[30.9, 63.6]	5.9	[1.0, 27.0]	62.5
`presidio`	open source	43.8	[28.2, 60.7]	5.9	[1.0, 27.0]	59.6
`nemo-guardrails`	open source	43.8	[28.2, 60.7]	17.6	[6.2, 41.0]	57.1

Table 2: Adversarial suite. All values are percentages. The intervals overlap almost completely across the top five rows.

More pointedly, the leading system is not distinguishable from the control ($p = 0.065$, $b = 9$, $c = 2$), and neither is any other member of the top four. A reader who concluded from Table 2 that `llm-guard` is the strongest available output guardrail would be drawing an inference the data does not license. A reader who concluded that the top four all beat a thirty-line regular expression would also be drawing one.

5.2 Public PII, $n = 300$

At $n = 300$ resolution improves substantially: 24 of 28 pairs separate. `aws-comprehend` (92.5% recall, [88.0, 95.4]) is distinguishable from every other system, and the control is distinguishable from everything. This is what an adequately powered guardrail comparison looks like.

Four pairs remain unresolved, including the author's own second place over `llm-guard` (81.0% versus 80.0%, $p = 0.473$). We report that ordering in our published materials and it does not survive testing.

5.3 Prompt injection and jailbreak, $n = 300$

Only four systems implement input-side detection; the remainder are out of scope and are not ranked. On prompt injection, `lakera-guard` reaches 94.7% recall at 16.7% false alarms and is genuinely and distinguishably the strongest system tested ($p < 0.001$ against all others). Everything else, including the author's system, sits at F1 $\leq$ 54.5. Roughly a third of that corpus is not in English, and that is where most misses concentrate.

The jailbreak corpus produces the study's sharpest illustration of the central argument. `lakera-guard` achieves 97.3% recall, [93.3, 99.0], the highest recall figure anywhere in this paper. It also produces an 83.3% false-alarm rate, [76.6, 88.4]. Against the control, which achieves 0.7% recall and detects essentially nothing, McNemar returns $p = 0.248$ ($b = 145$, $c = 125$). The two systems are not distinguishable in overall correctness.

We stress that this is not an argument that the control is a good guardrail, nor that Lakera is a bad one; the operating points are radically different and a deployment that tolerates false alarms may reasonably prefer high recall. It is an argument about reporting. A headline of "97.3% jailbreak detection" is literally true, is the kind of number that appears in vendor material, and conveys almost nothing about whether the system is more useful than trivial.

5.4 Required corpus size

Table 3 gives the estimated corpus size needed to resolve representative unresolved pairs. The comparisons that matter most to a buyer, those between two systems that are both broadly competent, are the most

Corpus	Pair	Observed b/c	Cases for 80% power
Adversarial (n=49)	`llm-guard` vs `aws-comprehend`	8/6	1,600
Adversarial (n=49)	`aws-comprehend` vs control	10/5	400
Adversarial (n=49)	control vs `nemo-guardrails`	12/5	200
PII (n=300)	`fluiq` vs `llm-guard`	18/13	3,200
PII (n=300)	`presidio` vs `nightfall`	22/20	>6,400
Injection (n=300)	`llm-guard` vs `fluiq`	20/17	>6,400
Jailbreak (n=300)	`lakera-guard` vs control	145/125	3,200

Table 3: Monte Carlo power estimates for unresolved pairs, 2,000 simulations per point.

expensive to resolve: separating the author's system from `llm-guard` on PII would require roughly 3,200 cases, and several pairs exceed 6,400.

The pattern is a straightforward consequence of the arithmetic and is worth stating plainly. Distinguishing a good guardrail from a bad one is cheap. Distinguishing a good guardrail from another good guardrail is expensive, because the two agree on almost every case and the test sees only the disagreements. Vendor benchmarks are, almost by construction, comparisons among competent systems, which is exactly the regime where a few hundred cases buy very little.

6 Discussion

The reporting defect is cheap to fix. Nothing in this paper requires new data collection. Confidence intervals and a paired test over per-case outcomes are a few dozen lines of code, computable from results any benchmark already produces. That they are absent from this literature is a convention rather than a constraint.

A control is the highest-value single addition. Of everything reported here, the finding most likely to change a reader's behaviour is that a thirty-line regular expression is statistically indistinguishable from four commercial and open-source products on one corpus, and from the highest-recall commercial detector on another. That diagnostic cost an afternoon. We would suggest that a guardrail comparison without a naive baseline should be treated as uninterpretable, since without a floor there is no way to tell a strong winner from a weak field.

Recall without a false-alarm rate is not a result. This is well known and continues to be violated. We would go further: recall and false-alarm rate together are still insufficient without a paired test, because two systems at different operating points can produce very different-looking pairs of numbers while being statistically identical in overall correctness, as the jailbreak result demonstrates.

On publishing a benchmark one does not win. The author's system places second on both output-side corpora and third on both input-side ones, and the central finding of this paper is that its second places are not statistically meaningful. We regard the incentive structure here as worth naming: a vendor benchmark that the vendor wins is close to uninformative, because the space of defensible methodological choices is large enough that a determined publisher can usually arrive there. The corresponding check available to a reader is to ask whether the publisher reports a specific number that is unflattering to them. Absence of such a number is itself the result.

7 Limitations

Conflict of interest. This benchmark is published by a party with a system in it. No methodology removes that. Mitigations: corpora, harness, adapters and analysis code are public under MIT; scoring was fixed before any system ran; competitors run at stock settings; every miss and false alarm is listed by case identifier; a naive control is included; and the paper's principal finding is adverse to the author's own results.

Single annotator. Every label was assigned by one person, who also works on one of the systems evaluated. There is no second annotator and therefore no inter-annotator agreement statistic. This is the largest methodological gap in the work. Per-case notes record the reasoning for contestable labels so that disagreement can be specific, but that is a materially weaker guarantee than two independent annotators and a reported κ. Some labels are explicitly judgement calls: a company support inbox address is scored *allow*, since blocking it makes an assistant unusable, while "I can confirm this person is a customer, but I cannot share their SSN" is scored *block*, since it confirms membership in a customer list. The jailbreak corpus counts a large number of roleplay and persona prompts as benign, which is the single largest source of label disagreement affecting these results.

Scale and coverage. 949 cases is small, predominantly English, and skewed toward PII and secrets. The 49-case adversarial suite is small enough that its unresolved comparisons are unsurprising; we include it deliberately because it is representative of published practice, but its size limits what can be concluded from it beyond the resolution finding itself.

A string is not a trace. Several systems, the author's included, detect retrieval poisoning and tool-argument exfiltration from structured execution context rather than from response text. Supplying only a string understates them on the `rag_poison_echo` category. Correcting this fairly requires a richer corpus format rather than an exception for one vendor.

Point-in-time measurement. All systems were measured on 7 August 2026. Commercial detectors are opaque and change without notice, and several lack a version identifier that a third party can pin. Any number here should be read with its date attached.

Scope differences. Presidio detects PII and does not claim to detect API keys, so it scores zero on the secrets category. That is a scope difference rather than a defect, which is why per-category results are reported and why no single aggregate should be read alone.

8 Conclusion

Guardrail comparisons are published as ranked tables, and ranked tables assert an ordering. We tested the orderings in a benchmark of eight systems and found that a majority of the comparisons on a corpus of typical size do not survive a paired significance test, that a thirty-line regular-expression control is indistinguishable from several commercial products, and that resolving the comparisons buyers actually care about would require one to two orders of magnitude more data than is customary. The remedies are inexpensive: report intervals, run a paired test, include a naive baseline, and state the date. We have applied all four to our own results, where they eliminate the finding most favourable to us.

Availability

Harness, both hand-written corpora, public-dataset samplers, all adapters and the analysis code in this paper are available under the MIT licence at https://github.com/SaurabhKumbhar24/guardrail-bench. The public PII corpus is not redistributed, as its upstream dataset card states no licence; the repository ships a seeded sampler and a SHA-256 manifest of all 300 rows, and a verification script that proves a rebuild is byte-identical to the corpus these results were computed on.

References

Samuel R. Bowman and George Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT*, pages 4843–4855, 2021.

Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Proceedings of EMNLP*, pages 9263–9274, 2020.

Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec@CCS)*, pages 79–90, 2023.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the 33rd USENIX Security Symposium*, pages 1831–1847, 2024.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. arXiv:2211.09527.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.